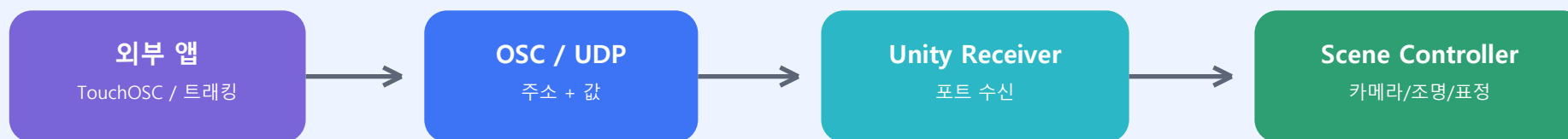




OSC 기초와 Unity 라이브 스튜디오 연동

Open Sound Control을 Unity 개발자 관점에서 이해하기



이 자료의 목표

OSC를 '외부 프로그램이 Unity 씬을 실시간으로 조작하는 언어'로 이해하고, VTuber/라이브 스튜디오 업무에 연결한다.

학습 산출물

카메라 전환, 조명 변경, 이펙트 On/Off, 표정 파라미터 제어를 OSC 메시지로 설계할 수 있게 된다.

프로젝트 기준: 왜 OSC를 배워야 하나

- 스텔라이브는 VTuber와 3D 라이브 콘텐츠를 다루는 회사이므로, Unity 씬이 외부 트래킹/운영 도구와 연결될 가능성이 높다.
- Brave group Unity 엔지니어 공고는 3D LIVE 시스템, 무대 연출, 조명, 부하 경감, 네트워크 지식을 업무/우대 요소로 다룬다.
- 현재 프로젝트는 Unity 6000.3.9f1 / URP 17.3.0 기반이고, manifest 기준 OSC 패키지는 아직 들어있지 않다.

항목	현재 프로젝트
Unity	6000.3.9f1
URP	17.3.0
Input System	1.18.0
UGUI	2.0.0
Cinemachine	3.1.7
OSC Package	없음

비유

URP가 무대의 조명/렌더링 설비라면, OSC는 조명 콘솔과 무대 장치를 연결하는 신호선에 가깝다.

실제 예시

오퍼레이터가 태블릿 버튼을 누르면 Unity에서 카메라 2번으로 전환되거나, 캐릭터 표정값이 0.8로 바뀐다.

OSC 한 문장 정의

정의

OSC는 컴퓨터, 사운드 신시사이저, 멀티미디어 장치 사이의 통신을 위해 만든 개방형 메시지 기반 프로토콜이다.

S1

Unity 관점

OSC는 Unity 내장 렌더 기능이 아니라 외부 앱에서 보낸 주소와 값을 받아 씬 상태를 바꾸는 통신 규칙이다.

비유

택배로 모델 파일을 보내는 것이 아니라, '조명 밝기 1.2', '카메라 3번' 같은 짧은 명령 쪽지를 빠르게 보내는 방식이다.

실제 예시

`/effect/bloom 1` 메시지를 받으면 Bloom 오브젝트를 켜고, `/camera/shot 2`를 받으면 2번 카메라 프리셋으로 전환한다.

Address

/light/intensity

Type Tags

,f

Arguments

1.25

그림: OSC 메시지를 강의용으로 단순화한 자체 제작 도식

OSC가 보내는 것과 보내지 않는 것

구분	OSC에 적합	OSC에 부적합
값	float, int, string, 작은 제어값	대용량 영상, 모델 파일, 긴 바이너리 파일
목적	상태 변경, 파라미터 제어, 트리거	실시간 영상 송출 자체, 에셋 배포
예시	<code>`/face/smile 0.8`, `/light/on 1`</code>	캐릭터 모델 전체 전송, 카메라 영상 스트림

비유

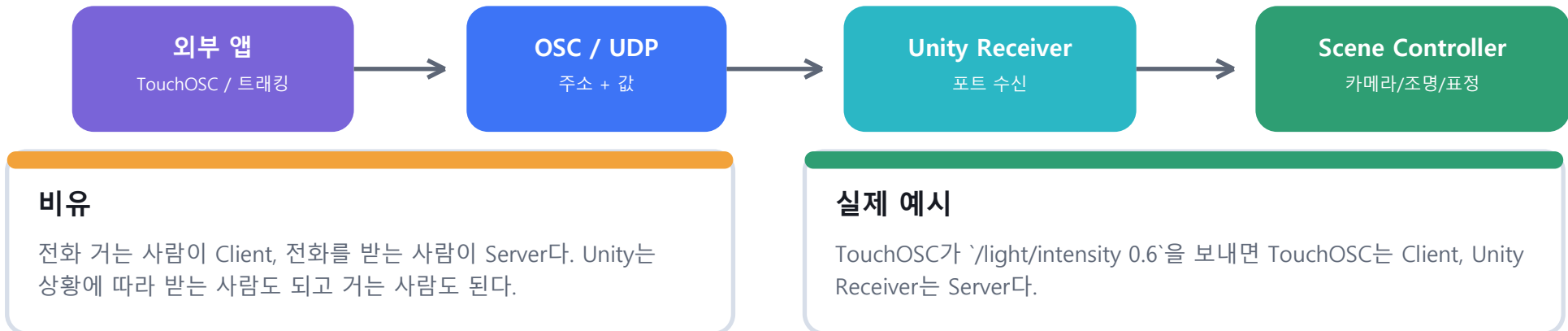
OSC는 물건을 실어 나르는 트럭이 아니라, 현장 스태프에게 보내는 무선 지시다.

실제 예시

표정, 눈 깜빡임, 손가락 포즈, 조명 색, 카메라 번호처럼 작은 숫자 데이터가 OSC와 잘 맞는다.

Client와 Server: 이름이 헷갈리는 지점

- OSC에서 Client는 메시지를 보내는 쪽이고, Server는 메시지를 받는 쪽이다.
- Unity가 외부 트래킹 앱의 데이터를 받는다면 Unity는 OSC Server 역할을 한다.
- Unity가 OBS나 조명 제어 앱으로 값을 보낸다면 Unity는 OSC Client 역할도 할 수 있다.



OSC 메시지 해부



그림: OSC 메시지를 강의용으로 단순화한 자체 제작 도식

부분	예시	의미
Address	<code>`/camera/shot`</code>	어떤 기능을 호출할지 나타내는 경로
Type Tag	<code>`i`</code>	뒤에 올 인자의 타입. i는 int32, f는 float32, s는 string
Arguments	<code>`2`</code>	실제로 전달할 값

실제 예시

``/camera/shot ,i 2``는 Unity에게 '카메라 샷 번호를 2로 바꿔라'라고 말하는 구조다.

Address 설계: 나중에 읽히는 이름이 중요하다

좋은 OSC Address 예시

```
/camera/shot 2  
/light/key/intensity 1.2  
/light/key/color 0.2 0.6 1.0  
/effect/bloom/enabled 1  
/avatar/expression/happy 0.8  
/stage/preset 3
```

규칙

OSC Address는 `/`로 시작하는 경로 구조다. URL처럼 읽히기 때문에 기능 단위로 계층화하면 운영자가 이해하기 쉽다.

S1

비유

폴더 경로를 정리하듯 주소를 정리한다. `/light/key/color`는 '조명 중 key light의 color'라는 뜻이 바로 보인다.

실제 예시

포트폴리오에서는 `/debug/fps`, `/camera/shot`, `/avatar/expression/\*`처럼 규칙을 문서화하면 실무 감각이 드러난다.

타입: 숫자 하나도 약속이 필요하다

태그	타입	Unity 예시
i	int32	카메라 번호, 프리셋 번호, On/Off 상태
f	float32	조명 밝기, 표정 가중치, 슬라이더 값
s	string	프리셋 이름, 상태 문자열, 캐릭터 이름
b	blob	임의 바이너리. 일반적인 라이브 제어에서는 남용하지 않는 편이 안전

비유

같은 '1'이라도 번호인지 밝기인지 On인지 약속이 없으면 현장 지시가 꼬인다.

실제 예시

`/avatar/expression/happy`는 float 0.0-1.0으로 정하고,
`/effect/bloom/enabled`는 int 0/1로 정한다.

UDP: 빠르지만 보장하지 않는다

핵심

OSC 패킷은 UDP 데이터그램으로 자연스럽게 표현될 수 있다.
VMC Protocol도 OSC over UDP/IP를 사용한다.

S1, S3

주의

UDP는 빠르고 단순하지만, 패킷 순서/도착을 항상 보장하는 방식으로 이해하면 안 된다. 수신부는 누락과 중복에 강해야 한다.

S8

비유

UDP는 등기우편이 아니라 무전이다. 빠르게 말하지만, 상대가 못 들었을 수도 있다고 생각해야 한다.

실제 예시

눈 깜빡임 값 한 프레임이 빠져도 다음 값이 오면 회복된다. 반면 '공연 시작' 같은 이벤트는 ACK/상태 확인을 별도로 설계하는 편이 좋다.

Bundle과 Timetag: 여러 명령을 묶는 봉투

- OSC Bundle은 `#bundle` 문자열과 timetag, 여러 bundle element로 구성된다.
- 같은 시점에 처리하고 싶은 메시지를 하나의 묶음으로 보낼 수 있다.
- 다만 OSC 자체가 시계 동기화를 제공하는 것은 아니므로, 실무에서는 송신/수신 앱의 시간 기준을 따로 고려한다.

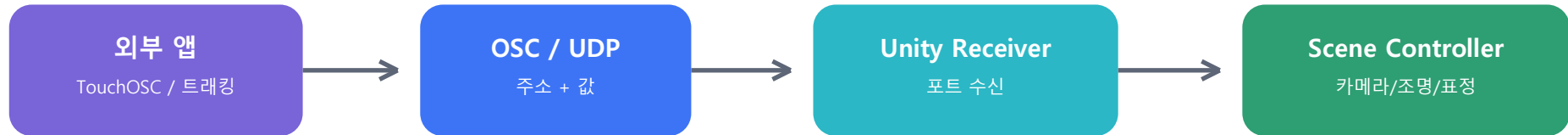
한 프레임에 같이 적용하고 싶은 값

```
#bundle timetag=now
/avatar/expression/happy 0.75
/avatar/expression/blinkLeft 0.0
/light/key/intensity 1.1
/camera/fov 35.0
```

비유

개별 쪽지를 따로 던지는 대신, 같은 장면 전환에 필요한 쪽지들을 한 봉투에 넣어 보내는 느낌이다.

Unity 수신 구조: 네트워크 스레드와 메인 스레드



중요한 구현 원칙

OSC 라이브러리가 별도 스레드에서 콜백을 호출하는 경우, Unity 오브젝트를 그 자리에서 직접 만지지 말고 큐에 넣은 뒤 Update에서 처리한다.

S6

권장 흐름 의사코드

```

OSC thread:
    queue.Enqueue(message)

Unity Update:
    while queue has message:
        ApplyToCameraOrLight(message)
  
```

실제 예시

수신 콜백에서 `light.intensity = value`를 바로 하지 않고, `pendingLightIntensity`에 저장한 뒤 Update에서 적용한다.

비유

무전실에서 받은 요청을 바로 무대에 뛰어가 처리하지 않고, 무대 감독의 큐시트에 올린 뒤 순서대로 처리하는 구조다.

Unity에서 쓸 수 있는 접근법

방법	장점	주의
OSC Jack	가볍고 Unity용 server/client 예제가 명확함	지원 타입과 스레드 처리 범위를 확인해야 함
extOSC	컴포넌트 기반 송수신/매핑/디버그 기능 제공	프로젝트 버전, 패키지 관리 방식, 유지보수 상태 확인
직접 구현	프로토콜 이해와 포트폴리오 설명력이 높음	패딩, 타입 태그, bundle 처리 등 구현 범위가 늘어남

실제 예시

처음 포트폴리오에서는 extOSC나 OSC Jack으로 빠르게 데모를 만들고, README에 메시지 주소 설계와 스레드 처리 방식을 설명한다.

실제 제어 예시: 카메라, 조명, 이펙트

운영용 OSC 주소 설계

```
/camera/shot 1  
/camera/fov 38.0  
/light/key/intensity 1.4  
/light/key/color 0.4 0.8 1.0  
/effect/bloom/enabled 1  
/effect/confetti/fire  
/stage/preset 2
```

업무 연결

공고문의 '오퍼레이터를 위한 시스템'은 이런 주소들을 UI 버튼/슬라이더와 연결해 실시간 제어 가능하게 만드는 일로 해석할 수 있다.

비유

카메라/조명/이펙트를 코드 속에 숨기지 않고, 외부 콘솔에서 누를 수 있는 버튼으로 꺼내는 작업이다.

실제 예시

태블릿에서 'Blue Stage' 버튼을 누르면 `/stage/preset 2`가 Unity로 들어오고, 켜진 조명색/배경/카메라를 한 번에 전환한다.

VMC Protocol: VTuber 쪽에서 특히 중요한 OSC 확장

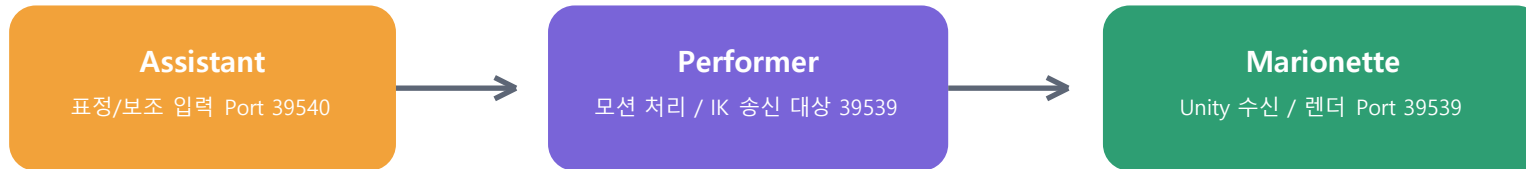


그림: VMC Protocol 용어를 Unity 개발자 관점으로 단순화한 자체 제작 도식

- VMC Protocol은 OSC와 VRM을 이용한 아바타 모션 통신 프로토콜이다.
- 기본 정보로 Bone 정보와 BlendShape 정보를 다룰 수 있고, 선택적으로 카메라, 라이트, 키보드, MIDI, 트래커, 시선 제어 등을 포함한다.
- VMC 문서는 모든 항목을 구현할 필요는 없고, 필요한 정보만 사용할 수 있다고 설명한다.

실제 예시

Unity가 Marionette 역할을 하면 외부 Performer 앱에서 보낸 `/VMC/Ext/Bone/Pos`나 표정 값을 받아 캐릭터를 움직인다.

VRM Expression: 표정값이 어디로 들어가나

VRM 요소	OSC/VMC와 연결되는 부분	Unity 구현 관점
Humanoid	본 이름과 포즈	Animator/HumanBodyBones로 매핑
Expression	표정 가중치 0.0-1.0	BlendShape 또는 MaterialColor 변경
LookAt	시선 방향/응시	눈 본 회전 또는 expression 기반 시선
SpringBone	머리카락/옷 흔들림	수신 포즈 적용 뒤 물리/스프링 계산

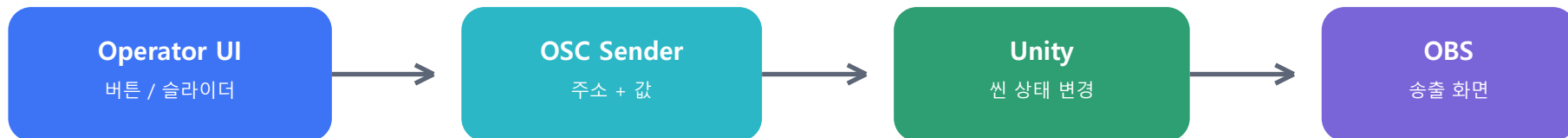
비유

OSC는 표정 버튼을 누르는 손이고, VRM Expression은 실제 캐릭터 얼굴 안쪽의 표정 슬롯이다.

실제 예시

`/avatar/expression/happy 0.8`을 받으면 VRM의 happy expression weight를 0.8로 보간해 적용한다.

오퍼레이터 패널 설계



- 오퍼레이터 UI는 비개발자가 눌러도 안전해야 한다.
- 모든 버튼은 어떤 OSC 주소를 보내는지 문서화한다.
- OBS는 자체 WebSocket 원격 제어 기능이 있으므로, Unity 제어는 OSC, OBS 제어는 WebSocket으로 분리하는 설계도 가능하다.

실제 예시

공연 전 'Preset Check' 버튼으로 카메라, 조명, 이펙트 상태를 초기화하고 Unity 화면과 OBS 장면을 맞춘다.

디버깅: OSC는 보이지 않아서 도구가 중요하다

확인 단계	질문	도구/방법
송신 확인	외부 앱이 실제로 보내나?	OSC Monitor, OSCDataMonitor, 앱 로그
네트워크 확인	IP/Port가 맞나?	같은 LAN, 방화벽, 포트 충돌 확인
수신 확인	Unity가 메시지를 받나?	Receiver 로그, 주소 전체 로깅
적용 확인	받은 값이 씬에 적용되나?	Update 큐, 상태 HUD, Frame Debugger

비유

무전이 안 들리면 먼저 송신기, 채널, 수신기, 현장 적용 순서로 나눠 봐야 한다.

실제 예시

`/light/key/intensity`가 안 먹으면 먼저 Unity 로그에 주소가 찍히는지 확인하고, 그 다음 Light 참조가 연결됐는지 본다.

안정성: 라이브 운영에서 조심할 것

- VMC Protocol은 안정적인 같은 로컬 네트워크 안에서의 사용을 전제로 하며, 인터넷을 넘는 사용은 전제로 하지 않는다고 설명한다.
- 알 수 없는 주소, 너무 많은 인자, 타입이 다른 인자는 무시하거나 안전하게 처리한다.
- Unity WebGL은 브라우저 제약 때문에 System.Net 네임스페이스의 .NET 네트워킹 클래스를 사용할 수 없다는 점도 기억한다.

방화벽

수신 포트가 차단되면 Unity는 아무 메시지도 받지 못한다.

스레드

수신 콜백에서 Unity 오브젝트를 직접 만지면 불안정할 수 있다.

주기

매 프레임 모든 메시지를 반드시 처리하려고 하면 프레임 드랍이 생긴다.

실제 예시

표정값은 최신 값만 유지하고 오래된 값은 버린다. 이벤트성 버튼은 중복 실행 방지 시간을 둔다.

실습 1: OSC로 Unity 씬 제어하기

- 목표: 외부 OSC Sender에서 보낸 값으로 Unity 카메라, 조명, 이펙트를 제어한다.
- 구성: `OSCReceiver`, `LiveSceneController`, `CameraPreset`, `LightPreset`, `DebugHUD`.
- 측정: 수신 메시지 수, 마지막 수신 시간, 적용된 현재 상태를 화면에 표시한다.

최소 메시지 목록

```
/camera/shot 1  
/light/key/intensity 1.0  
/effect/bloom/enabled 1  
/stage/preset 2  
/debug/ping 123.0
```

실제 예시

시연 영상에는 Sender 화면, Unity Game View, DebugHUD가 동시에 보이게 녹화한다.

실습 2: OSC로 캐릭터 표정 제어하기

- 목표: `/avatar/expression/{name}` 주소로 받은 0.0-1.0 값을 캐릭터 표정에 적용한다.
- VRM 모델이 있으면 VRM Expression으로 연결하고, 없으면 임시 SkinnedMesh BlendShape로 대체한다.
- 보간, 값 clamp, 주소 미등록 처리, 마지막 수신 시간 표시를 구현한다.

표정 주소 예시

```
/avatar/expression/happy 0.8  
/avatar/expression/angry 0.2  
/avatar/expression/blink 1.0  
/avatar/expression/aa 0.4  
/avatar/lookat 0.2 -0.1
```

비유

OSC는 리모컨이고, Expression은 캐릭터 얼굴 안의 슬라이더다.
리모컨 값이 슬라이더 위치를 바꾼다.

포트폴리오에 남길 증거

산출물	보여줄 역량	포트폴리오 설명 문장
OSC Control Demo	외부 제어 연동	OSC 메시지로 Unity 라이브 씬을 제어했습니다.
Message Spec 문서	운영/협업 문서화	주소, 타입, 값 범위, 실패 처리를 명세했습니다.
Debug HUD	운영 안정성	마지막 수신 메시지와 현재 상태를 현장에서 확인할 수 있습니다.
Failure Test	QA/유지보수	포트 오류, 타입 오류, 패킷 누락 상황을 테스트했습니다.

실제 예시

README에는 '왜 OSC를 썼는지', '어떤 주소를 설계했는지', '수신 스레드를 어떻게 메인 스레드로 넘겼는지'를 적는다.

OSC 학습 로드맵

1. OSC 메시지 구조

Address / Type Tag / Argument

2. Unity 수신

포트 열기, 메시지 로그

3. 씬 제어

카메라, 조명, 이펙트

4. 캐릭터 제어

Expression, LookAt

5. 운영 안정성

큐, 방화벽, 타입 검증

6. 문서화

주소 명세, 테스트표, 시연 영상

비유

처음부터 모션 캡처 전체를 만들 필요는 없다. 먼저 버튼 하나가 Unity 씬을 바꾸는 통로를 확실히 만든다.

면접/지원서에서 말할 수 있는 포인트

- OSC는 Unity와 외부 운영/트래킹 도구를 연결하는 메시지 기반 프로토콜로 이해하고 있습니다.
- 라이브 환경에서는 빠른 제어보다도 값 검증, 누락 허용, 상태 표시, 복구 루틴이 중요하다고 보고 설계했습니다.
- VMC Protocol은 OSC와 VRM을 활용하므로, 캐릭터 포즈/표정/카메라/라이트 같은 데이터를 선택적으로 다룰 수 있습니다.
- 포트폴리오에서는 카메라/조명/이펙트/표정 제어를 각각 OSC 주소로 분리해 문서화했습니다.

실제 예시

좋은 답변: 'OSC를 단순 통신 라이브러리만으로 보지 않고, 운영자가 누르는 버튼과 Unity 씬 상태 사이의 명세로 관리했습니다.'

최종 체크리스트

항목	완료 기준
☐ 주소 명세 작성	주소, 타입, 값 범위, 기본값, 실패 처리 문서화
☐ Unity Receiver 구현	포트 수신, 로그, 큐, 메인 스레드 적용
☐ 오퍼레이터 패널 연결	버튼/슬라이더가 의도한 OSC 메시지 전송
☐ 표정/카메라/조명 데모	최소 3종류 이상의 씬 제어 확인
☐ 오류 상황 테스트	포트 오류, 잘못된 타입, 미등록 주소, 과다 메시지
☐ 문서/영상	README, 메시지표, 시연 영상, 트러블슈팅 기록

출처 1

ID	문서	기관/저자	URL
S1	OpenSoundControl Specification 1.0	OpenSoundControl.org / Matt Wright	https://opensoundcontrol.stanford.edu/spec-1_0.html
S2	VMC Protocol specification	VirtualMotionCaptureProtocol	https://protocol.vmc.info/english.html
S3	VMC Protocol specification overview	VirtualMotionCaptureProtocol	https://protocol.vmc.info/specification.html
S4	VRM 1.0 specification	VRM Consortium / vrm-c	https://github.com/vrm-c/vrm-specification/blob/master/specification/VRMC_vrm-1.0/README.md
S5	VRM 1.0 expressions specification	VRM Consortium / vrm-c	https://github.com/vrm-c/vrm-specification/blob/master/specification/VRMC_vrm-1.0/expressions.md
S6	OSC Jack	Keijiro Takahashi	https://github.com/keijiro/OscJack
S7	extOSC	lam1337	https://github.com/lam1337/extOSC

출처 2

ID	문서	기관/저자	URL
S8	UdpClient class	Microsoft Learn	<a href="https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.udpc
lient">https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.udpc lient
S9	Unity WebGL networking	Unity Technologies	https://docs.unity.cn/Manual/webgl-networking.html
S10	OBS Remote Control Guide	OBS Project	https://obsproject.com/kb/remote-control-guide
S11	STELLIVE ABOUT	STELLIVE	https://stellive.me/about
S12	Brave group Unity Engineer job posting	Brave group	https://hrmos.co/pages/bravegroup/jobs/000100100011
P1	Project Unity version	local project file	ProjectSettings/ProjectVersion.txt
P2	Project package manifest	local project file	Packages/manifest.json